

Statistical Analysis of Experimental Results The Tournament

Brandon Marquez Salazar & Cindy Ortiz Escoto

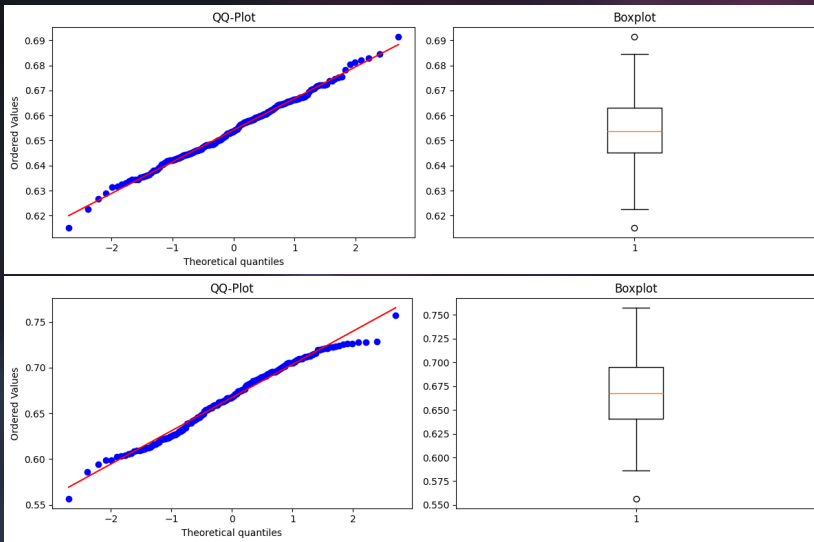
Curso de Estadística Inferencial

Normality Test Results - Shapiro-Wilk Test

Dataset	Statistic (W)	P-value	Normal
Dataset de Brandon	0.996 800	0.955 533	True
Dataset de Cindy	0.983 705	0.020 487	False

Table: Shapiro-Wilk normality test results. Null hypothesis (H_0): Data follows a normal distribution.

Normality Test Results



Dataset Overview - Descriptive Statistics

Dataset	Samples	Mean	Median	Std
Dataset de Brandon	200	0.654 155	0.653 683	0.012 535
Dataset de Cindy	200	0.667 328	0.667 888	0.036 239

Dataset	Minimum	Maximum
Dataset de Brandon	0.615 156	0.691 413
Dataset de Cindy	0.556 569	0.757 423

Table: Descriptive statistics for both datasets showing central tendency and dispersion measures.

Variance Test - Levene Test Results

Method	Statistic	P-value	Equal Variance
Levene	167.485 723	$3.222\,765 \times 10^{-32}$	False

Table: Levene test for equality of variances between datasets. The extremely small p-value indicates significant difference in variances.

Statistical Difference Test - Wilcoxon Rank-Sum Test

Method	Statistic	P-value	Equal Distributions
Wilcoxon	14 466.0	$1.699\,915 \times 10^{-6}$	False

Table: Wilcoxon rank-sum test results. The small p-value indicates statistically significant difference between the two datasets.

Complete Statistical Analysis Summary

Test Type	Dataset 1 Result	Dataset 2 Result	Test Statistic	Conclusion
Normality (Shapiro-Wilk)	Normal ($p=0.9555$)	Not Normal ($p=0.0205$)	$W=0.9968/0.9837$	Mixed
Variance (Levene)	Variances not equal		$F=167.49$	He
Difference (Wilcoxon)	Distributions different		$W=14466.0$	Sta

Key Findings

- ▶ Dataset de Brandon follows normal distribution
- ▶ Dataset de Cindy does not follow normal distribution
- ▶ Variances between datasets are significantly different
- ▶ The two datasets show statistically significant differences
- ▶ Non-parametric Wilcoxon test was appropriate due to violated assumptions

Statistical Analysis Pseudocode - Main Functions

```
1: function STATISTICAL_ANALYSIS(opts)
2:   res["normality_test"][opts.ds_01_name] ← Normality_Test(opts.ds_01)
3:   res["overview"][opts.ds_01_name] ← DatasetOverview(opts.ds_01)
4:   if ds_02 exists then
5:     res["normality_test"][opts.ds_02_name] ← Normality_Test(opts.ds_02)
6:     res["overview"][opts.ds_02_name] ← DatasetOverview(opts.ds_02)
7:     res["variance_test"] ← Variance_Test(opts, res)
8:     res["difference_test"] ← Statistical_Difference_Test(opts, res)
9:   end if
10:  return res
11: end function
12: function NORMALITY_TEST(ds)
13:  out ← Shapiro-Wilk(ds)
14:  out["method"] ← "Shapiro-Wilk"
15:  if out["p-value"] < 0.05 then
16:    out["normal"] ← false
17:  else
18:    out["normal"] ← true
19:  end if
20:  return out
21: end function
```


Statistical Analysis Pseudocode - Support Functions

```
1: function DATASETOVERVIEW(ds)
2:   out["samples"] ← length(ds)
3:   out["mean"] ← mean(ds)
4:   out["median"] ← median(ds)
5:   return out
6: end function
7: function VARIANCE_TEST(opts, res)
8:   if both datasets normal then
9:     out ← F-Test(opts.ds_01, opts.ds_02)
10:    out["method"] ← "F-Test"
11:   else
12:     out ← Levene(opts.ds_01, opts.ds_02)
13:     out["method"] ← "Levene"
14:   end if
15:   if out["p-value"] < 0.05 then
16:     out["equal"] ← false
17:   else
18:     out["equal"] ← true
19:   end if
20:   return out
21: end function
```

Statistical Analysis Pseudocode - Difference Test

```
1: function STATISTICAL_DIFFERENCE_TEST(opts, res)
2:   if both normal and equal variance then
3:     out["alpha_grid"] ← alpha_optimizer(opts.ds_01, opts.ds_02, opts)
4:     out ← T-Test(opts.ds_01, opts.ds_02, best_alpha)
5:     out["method"] ← "T-Test"
6:     if out["p-value"] < best_alpha then
7:       out["equal"] ← false
8:     else
9:       out["equal"] ← true
10:    end if
11:  else
12:    out ← Wilcoxon(opts.ds_01, opts.ds_02)
13:    out["method"] ← "Wilcoxon"
14:  end if
15:  return out
16: end function
```

Statistical Analysis Pseudocode - Alpha Optimizer

```
1: function ALPHA_OPTIMIZER(x, y, opts)
2:   alpha_list ← opts["alpha"]["values"] or default
3:   P ← opts["alpha"]["P"] or 0.4
4:   k ← opts["alpha"]["k"] or 1
5:   sample_size ← opts["alpha"]["sample_size"] or 6
6:   iterations ← opts["alpha"]["iterations"] or 500
7:   for each alpha in alpha_list do
8:     for i in 1 to iterations do
9:       sample_x ← sample(x, sample_size, replace=True)
10:      sample_y ← sample(y, sample_size, replace=True)
11:      pvalue ← t.test(sample_x, sample_y)$p.value
12:      update rejection_count
13:    end for
14:    power ← rejected / total
15:    beta ← 1 - power
16:    expected_loss ← (P * alpha) + ((1 - P) * k * beta)
17:  end for
18:  best_alpha ← alpha with min(expected_loss)
19:  return results with best_alpha
20: end function
```